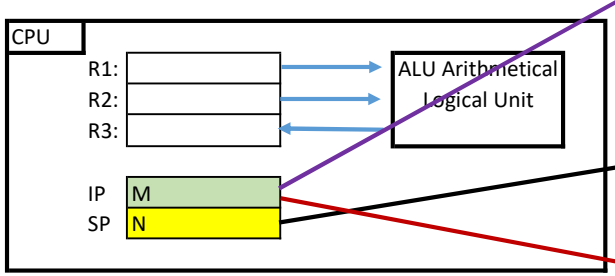
Calling function F(v)

1 IP points to current instruction: Call function F(X) with parameter v

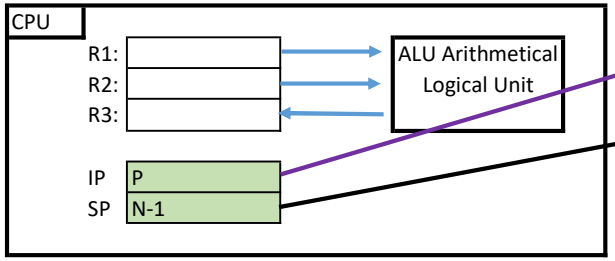
Adresse	0	1	...	Variable v	...		M	M+1		P	P+1	...	P+k		N-2	N-1	N
Memory:				123			Call Funct F(X) with parameter v	...		code for func F()X	...	...	return form Function F(X)				



Programm execution just encountered the call of a function/method F(X)
at address M
Please note: SP points to the next available cell.
Also, the stack SP goes down(!)

2 IP: Prepare call - save current IP on stack

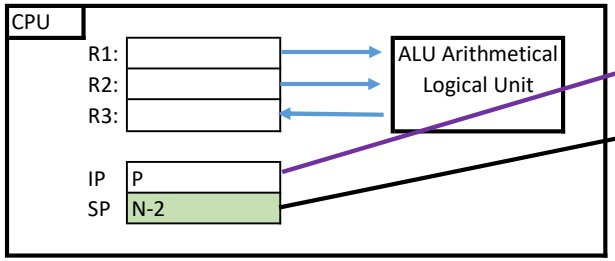
Adresse	0	1	...	Variable v	...		M	M+1		P	P+1	...	P+k		N-2	N-1	N
Memory:				123			Call Funct F(X) with parameter v	...		code for func F()X	...	...	return form Function F(X)				M+1



Start "Stack Discipline":
Store current IP address on the stack(!) - "bookmark"
we want to now where to continue after function call ...
SP is decremented(!), stack goes down
SP points to the *next available* position on stack
IP gets the start instruction of the function(!)

3 IP: Prepare call - save *value of* function parameter v on stack

Adresse	0	1	...	Variable v	...		M	M+1		P	P+1	...	P+k		N-2	N-1 also param v for F!	N
Memory:				123			Call Funct F(X) with parameter v	...		code for func F()X	...	...	return form Function F(X)			123	M+1

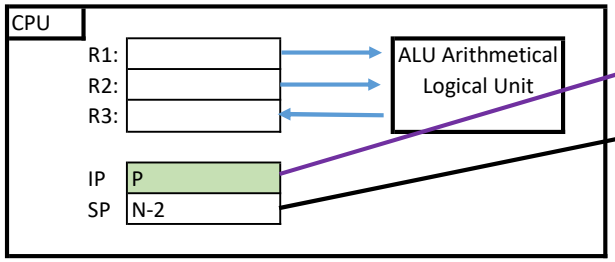


The "call frame"

Still "Stack Discipline":
Store value of parameter on stack for the function itself
Pls note: call by value!
The stack part for the function/method call is named "the call frame"
The function code for F(X) has no clue where the original value v is stored ...

4 IP points to current instruction: Start executing code at function F(X)

Adresse	0	1	...	Variable v	...		M	M+1		P	P+1	...	P+k		N-2	N-1 also param v for F!	N
Memory:				123			Call Funct F(X) with parameter v	...		code for func F()X	...	...	return form Function F(X)			123	M+1



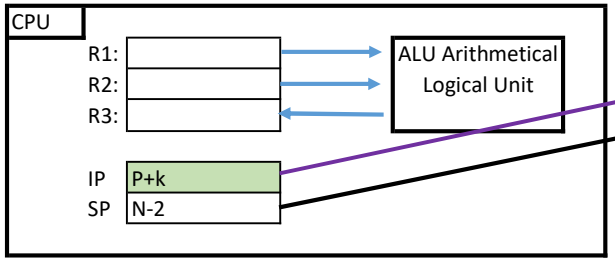
The "call frame"

Programm execution is now "business as usual"
The code in the function works on the address N-1 as formal parameter(!) v
The function operates on the "call frame"
The function code for F(X) has no clue where the original value v is stored ...

5 IP: Function executions reaches return statement (IP moves)

Adresse	0	1	…	Variable v	…		M	M+1		P	P+1	…	P+k		N-2	N-1 also param v for F!	N
Memory:				123			Call Funct F(X) with parameter v	…		code for func F()X	…	…	return form Function F(X)			123	M+1

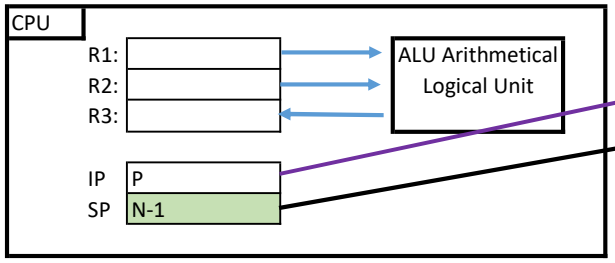
The "call frame"



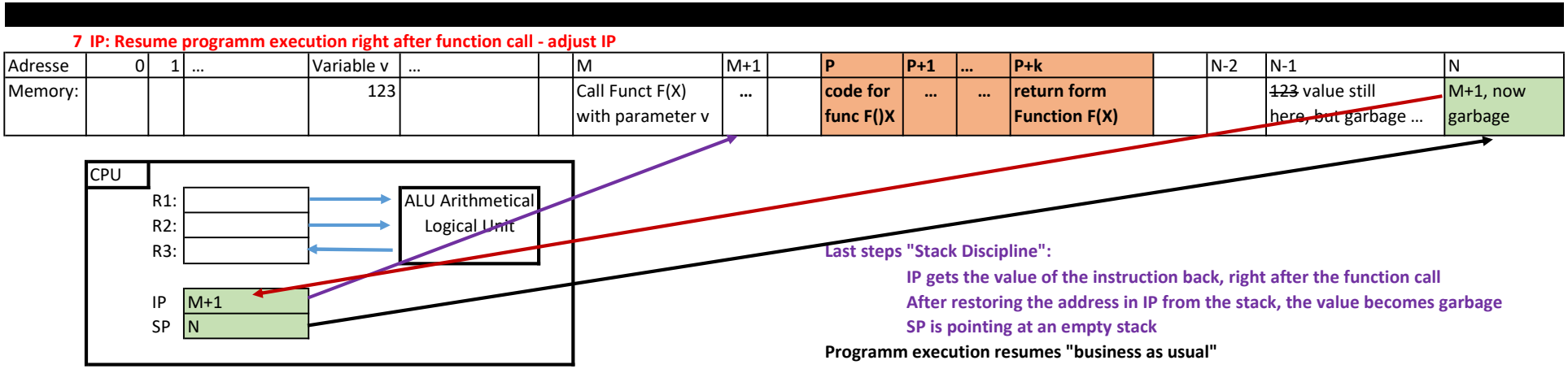
Funtion/method has finished execution, here: regularly.
Same applies for every return statement.
Every return triggers continuation of "Stack Discipline"
Even more "Stack Discipline":
The sequence for restoring the original programm flow commences
The call frame will be destroyed, not needed anymore.

6 IP: The function frame on the stack is destroyed simply by adjusting the SP

Adresse	0	1	…	Variable v	…		M	M+1		P	P+1	…	P+k		N-2	N-1	N
Memory:				123			Call Funct F(X) with parameter v	…		code for func F()X	…	…	return form Function F(X)			~~123~~ value still here, but garbage …	M+1



Yet more "Stack Discipline":
After destroying the function call frame, SP points to the "old" address, for the programm to execute
Since SP points to the next free place on the stack, the old IP address is found in stack address N



What is "Stack Discipline"?

The above interaction of IP and SP whenever a function/method call is processed.
If the function/method has a return object, this is left on the stack(!) to be collected by the original call.
Please note: Using the stack allows a function/method call to further call other functions/methods.

What is the frame of a function/method call?

The stack portion dedicated to the call of the function/method. It is a static place on the stack during the call.
As part of the "Stack Discipline" the frame is
- allocated during the start of call,
- used by the function/method for local variables,
- destroyed after the call is finished.

Please note: The parameters of the function/method, as well as the local variables, are allocated on the stack(!).

Brain teaser: A function/method can call itself(!) - how does the "Stack Discipline" look like and how the frames?

It's the stack & "Stack Discipline" who does the actual work by guaranteeing the frames … , everything else is "business as usual".